

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo E (azul).
Profesor: Isabel Pita.

Nombre del alumno:

1. Dadas las siguientes funciones de coste, indica su orden de complejidad (representado por la función más sencilla) y ordena los órdenes de complejidad obtenidos utilizando $\subset$.

a) $f_1(n) = 5n^2 + 3n + 1$

b) $f_2(n) = n + \log n + 3$

c) $f_3(n) = n^5 + 3n^3 + 2$

e) $f_4(n) = \log_2(n + 3) + 5$

d) $f_5(n) = \log_3 n$

2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n >> k;
for (int i = 0; i < n; ++i){
    for (int j = 0; j < k; ++j) ++x;
    for (int j = k; j < n; ++j) --x;
}
```

3. Escribe un predicado para expresar que los valores de un vector v , entre las posiciones i y j son positivos.
4. Escribe un predicado para expresar que el número de valores pares de un vector v entre las posiciones i y j es mayor que un cierto valor x .
5. Especifica un algoritmo que dado un vector de números enteros, cuya longitud es mayor que cero, calcule la longitud de la secuencia más larga de números positivos tal que más de la mitad de los valores de la secuencia sean números pares.

6. Dada la especificación:

$P: \{v.size() \geq 0\}$

$\text{resolver}(\text{vector}<\text{int}> v) \text{ dev } \text{vector}<\text{int}> \text{ sol}$

$Q: \{\forall x: 0 \leq x < \text{sol.size()} : \text{sol}[x] = \sum k : x \leq k < v.size() : v[k]\}$

Y dada la siguiente implementación:

```
std::vector<int> resolver(std::vector<int> const& v) {
    std::vector<int> sol(v.size());
    sol[v.size()-1] = v[v.size()-1];
    for (i = v.size()-1; i > 0; --i) {
        sol[i-1] = v[i-1] + sol[i];
    }
    return sol;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo:

7. Dado un vector de números enteros cuyos elementos están ordenados en orden decreciente, indica un algoritmo *óptimo* que permita obtener el máximo del vector.

8. Define una función recursiva que resuelva el problema de contar el número de veces que aparece el dígito 1 en un número. Observad que se pide la función recursiva.
9. Indica una recurrencia que permita calcular el coste en el caso peor del siguiente algoritmo, haz el desplegado y obtén el orden de complejidad del algoritmo.

```
bool f ( std::vector<int> const& v, int ini, int fin, int x){  
    if (ini == fin) return true; // Vector vacio  
    else {  
        int m = (ini + fin - 1) / 2;  
        if (v[m] > v[ini]) return f(v,ini,m,x);  
        else if (v[m] <= v[ini] && x < v[m]) return f(v,m,fin,x);  
        else return f(v, ini, m+1,x) || f(v,m,fin,x);  
    }  
}
```

10. Indica cuál es la recurrencia que permite calcular el coste del algoritmo quicksort en el caso peor y en el caso medio. Indica el orden de complejidad del algoritmo en ambos casos.